

Aplicación de estilos y responsividad

Módulo: Fundamentos de desarrollo Front-End

| AE3: Aplicar hojas de estilo CSS básicas distinguiendo elementos de responsividad para personalizar la presentación de un documento HTML acorde a un requerimiento entregado

Introducción

El diseño y la presentación visual de una página web son aspectos fundamentales en el desarrollo frontend, y para dominarlos es esencial comprender la aplicación de estilos y la responsividad. Este manual te guiará en el uso de CSS, el lenguaje que permite definir la apariencia de los sitios web, y en las mejores prácticas para estructurar y organizar estilos de manera eficiente. Además, conocerás cómo optimizar el manejo de imágenes y otros recursos, asegurando un diseño visual atractivo y bien estructurado.

En un entorno digital donde la variedad de dispositivos es cada vez mayor, la responsividad se ha convertido en un estándar imprescindible. Aquí aprenderás cómo adaptar el contenido para diferentes tamaños de pantalla, priorizando la experiencia de usuario con enfoques modernos como Mobile First y técnicas como Media Queries. También explorarás herramientas para inspeccionar y depurar estilos, facilitando el desarrollo de interfaces flexibles y accesibles.

Un conocimiento sólido en estilos y responsividad te permitirá crear sitios web más eficientes, accesibles y visualmente atractivos, diferenciándote en un mercado que exige habilidades técnicas cada vez más especializadas.

Aprendizaje esperado

Cuando finalices la lección podrás:

- Entender la importancia de las hojas de estilo CSS en el diseño y la presentación de sitios web y la diferencia entre HTML y CSS, y cómo trabajan juntos para crear sitios web visualmente atractivos.
- Aprender cómo utilizar selectores y reglas CSS para aplicar estilos a elementos específicos de una página web.
- Comprender las propiedades CSS más comunes y cómo se utilizan para controlar aspectos como colores, fuentes, tamaños, márgenes y diseños.
- Crear diseños responsivos utilizando CSS para adaptar un sitio web a diferentes tamaños de pantalla y dispositivos.
- Comprender los beneficios de utilizar hojas de estilo CSS, como la separación de la presentación del contenido, la facilidad de mantenimiento y la mejora de la usabilidad y accesibilidad.

Hoja de estilos CSS

Las hojas de estilo CSS (Cascade Style Sheets) son un componente esencial en el desarrollo de sitios web modernos. Estas hojas de estilo, escritas en lenguaje CSS, son utilizadas para definir y controlar la apariencia visual de un sitio web, incluyendo elementos como colores, fuentes, márgenes, tamaños y diseños. La cascada en el nombre "Cascade Style Sheets" se refiere al modo en que se aplican los estilos en CSS. Cuando un navegador lee un archivo HTML, también busca y lee las hojas de estilo CSS asociadas. Si hay varias reglas CSS que afectan a un mismo elemento HTML, se produce un proceso de cascada donde se aplican diferentes estilos según su relevancia y jerarquía.

Las hojas de estilo CSS **permiten tener un control preciso sobre la apariencia de un sitio web**, ya que se pueden definir estilos de forma global para afectar a todos los elementos de una página, o bien establecer reglas específicas para elementos individuales o grupos de elementos. Además, CSS ofrece numerosas ventajas a los desarrolladores y diseñadores web. Permite separar la presentación visual del contenido estructural, lo que facilita la mantenibilidad y la actualización de un sitio web. También proporciona flexibilidad y adaptabilidad, ya que se pueden crear diseños responsivos que se ajustan automáticamente a diferentes tamaños de pantalla y dispositivos.

Con el uso de hojas de estilo CSS, es posible crear diseños atractivos, coherentes y profesionales, mejorar la legibilidad del contenido, optimizar la usabilidad y la accesibilidad, y brindar una experiencia visualmente agradable a los visitantes del sitio.

Características de las Hojas de Estilo

El manejo de hojas de estilo CSS implica una serie de características clave que te permiten diseñar y controlar la apariencia de un sitio web de manera efectiva. Algunas de estas características son:

- ★ **Separación de la presentación del contenido:** CSS permite separar el diseño y la presentación visual del contenido estructural de un sitio web. Esto significa que puedes mantener el HTML limpio y centrado en la estructura y el significado del contenido, mientras que los estilos CSS se encargan de definir cómo se ve ese contenido.
- ★ **Reglas y selectores:** En CSS, se puede utilizar reglas y selectores para aplicar estilos a elementos específicos de una página web. Los selectores

te permiten identificar los elementos HTML a los que deseas aplicar un estilo, y las reglas definen los estilos que deseas aplicar a esos elementos.

- ★ **Cascada y herencia:** CSS sigue un modelo de cascada, lo que significa que los estilos pueden heredarse de **elementos padre a elementos hijos**. Esto te permite establecer estilos generales en un nivel superior y luego anular o agregar estilos específicos en niveles inferiores. Además, se puede utilizar la propiedad "inherit" para heredar explícitamente estilos de un elemento padre.
- ★ **Propiedades y valores:** CSS ofrece una amplia gama de propiedades y valores que se pueden utilizar para controlar aspectos visuales como colores, fuentes, tamaños, márgenes, rellenos, bordes, alineaciones y diseños. Cada propiedad tiene un conjunto específico de valores que determinan cómo se aplicará ese estilo.
- ★ **Unidades de medida:** CSS proporciona diferentes unidades de medida que puedes utilizar para especificar tamaños y dimensiones. Algunas unidades comunes incluyen píxeles (px), porcentajes (%), em, rem, vh y vw. Estas unidades te permiten adaptar los estilos a diferentes tamaños de pantalla y dispositivos.
- ★ **Media queries:** Las media queries son una característica clave de CSS que te permite crear diseños responsivos y adaptativos. Con las media queries, se puede definir estilos específicos que se aplicarán según el tamaño de la pantalla o el dispositivo en el que se visualiza el sitio web. Esto te permite optimizar la experiencia del usuario en diferentes dispositivos.
- ★ **Estilos en línea, incrustados y externos:** CSS se puede utilizar de diferentes formas. Se puede aplicar estilos directamente en línea dentro de los elementos HTML, utilizar estilos incrustados en la sección <style> del HTML o enlazar un archivo CSS externo utilizando la etiqueta <link>. Cada enfoque tiene sus ventajas y se elige según las necesidades y preferencias del proyecto.

CSS, fundamentos y utilidad

CSS (Cascading Style Sheets) es un lenguaje de estilo utilizado para controlar y definir la apariencia visual de los documentos HTML y XML. Es un estándar web ampliamente utilizado y una parte fundamental en el desarrollo de sitios web modernos. Los fundamentos de CSS se centran en la separación de la

presentación visual del contenido estructural. Mientras que **HTML se utiliza para marcar y estructurar el contenido** de una página web, **CSS se encarga de definir cómo se ve ese contenido**. Esto permite una clara separación de preocupaciones, lo que facilita el mantenimiento y la actualización del sitio web.

CSS funciona aplicando **reglas de estilo** a los elementos HTML mediante selectores. Un selector selecciona los elementos específicos a los que se aplicará un estilo y las reglas definen cómo se verán esos elementos. Por ejemplo, **se puede utilizar un selector de clase para seleccionar todos los elementos con una clase determinada y aplicarles un estilo específico**. Las reglas CSS están compuestas por propiedades y valores. Las propiedades representan los aspectos visuales que deseas controlar, como el color, la fuente o el tamaño del texto, los márgenes, los bordes, entre otros. Los valores, por su parte, especifican cómo se aplicará esa propiedad en particular.

CSS también tiene un modelo de cascada, lo que significa que los estilos se aplican en función de su relevancia y jerarquía. Si hay varias reglas que afectan a un mismo elemento, **se aplicará la regla más específica** y prioritaria. Esto permite tener un mayor control y flexibilidad en la aplicación de estilos.

CSS y HTML

CSS (Cascading Style Sheets) y HTML (HyperText Markup Language) son dos lenguajes fundamentales para el desarrollo de sitios web. Aunque son diferentes en su naturaleza y propósito, trabajan en conjunto para crear una experiencia web completa.

HTML es un lenguaje de marcado utilizado para estructurar y organizar el contenido de una página web. Con HTML, se definen los elementos como encabezados, párrafos, listas, imágenes, enlaces y más. Estos elementos proporcionan la base estructural del sitio web y determinan cómo se presenta y se relaciona el contenido.

Por otro lado, **CSS es un lenguaje de estilo utilizado para controlar la apariencia visual de los elementos HTML**. Mientras que HTML se enfoca en la estructura y el significado del contenido, **CSS se encarga de definir cómo se verá ese contenido en el navegador**. Con CSS, se pueden establecer estilos como colores, fuentes, tamaños de texto, márgenes, espaciados y diseños para los elementos HTML.

La relación entre CSS y HTML es muy estrecha. HTML proporciona la estructura y el contenido del sitio web, mientras que CSS se encarga de la presentación y el estilo de ese contenido. Juntos, permiten crear páginas web visualmente atractivas y funcionales.

En términos prácticos, **la forma de utilizar CSS y HTML en conjunto es a través de la vinculación de archivos.** Se crea un archivo HTML que contiene la estructura del sitio web, y se enlaza un archivo CSS separado que contiene las reglas de estilo. La etiqueta <link> se utiliza para establecer la conexión entre el archivo HTML y el archivo CSS.

Estilos en línea, embebidos, archivos externos

En CSS, **existen tres formas** principales de incluir estilos en un documento HTML: **estilos en línea, estilos embebidos y estilos en archivos externos.**

Estilos en línea: Los estilos en línea se aplican directamente a los elementos HTML mediante el uso del atributo style. Por ejemplo, puedes aplicar estilos en línea a un párrafo de la siguiente manera:

```
<p style="color: red; font-size: 16px;">Este es un párrafo  
con estilos en línea.</p>
```

En este caso, el color del texto se establece como rojo y el tamaño de la fuente como 16 píxeles.

Los estilos en línea se aplican directamente al elemento y tienen prioridad sobre otros estilos definidos en estilos embebidos o archivos externos. Sin embargo, pueden volverse difíciles de mantener si se aplican a varios elementos o si se necesita realizar cambios frecuentes.

Estilos embebidos: Los estilos embebidos se definen dentro de la sección <style> en el encabezado del documento HTML. Esta técnica permite agrupar los estilos relacionados en un solo lugar. Por ejemplo:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <style>
```

```

    p {
      color: blue;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <p>Este es un párrafo con estilos embebidos.</p>
</body>
</html>

```

En este ejemplo, los estilos para los párrafos se definen dentro de la **etiqueta <style>**. Los estilos se aplicarán a todos los **elementos <p>** en el documento.

Los estilos embebidos tienen precedencia sobre los estilos definidos en archivos externos, pero los estilos en línea tienen prioridad sobre los estilos embebidos.

Archivos externos de CSS: Esta es la forma más común y recomendada de incluir estilos en un documento HTML. En lugar de definir los estilos directamente en el documento HTML, **se crean archivos CSS** separados con reglas de estilo y luego **se enlazan al documento HTML** utilizando la **etiqueta <link>**. Por ejemplo:

En el archivo **styles.css**:

```

p {
  color: green;
  font-size: 20px;
}

```

En el archivo **index.html**:

```

<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <p>Este es un párrafo con estilos desde un archivo
externo de CSS.</p>
  </body>
</html>

```

En este último, **los estilos se definen en el archivo styles.css y se vinculan al documento HTML utilizando la etiqueta <link>**. Los estilos se aplicarán a todos los elementos <p> del documento.

Los archivos externos de CSS proporcionan una mayor organización y mantenibilidad, ya que los estilos se pueden reutilizar en múltiples páginas HTML y se pueden actualizar fácilmente sin tener que editar cada archivo HTML individualmente.

Es SUPER importante poder dividir los lenguajes: tener el html en un archivo, y el css en otro archivo. De esta manera todo queda más limpio y ordenado. La relación es “stylesheet” que quiere decir “hoja de estilos”.

Referencias y selectores, por clase, por id

Los selectores en CSS son utilizados para seleccionar elementos específicos dentro de un documento HTML y aplicar estilos a esos elementos. Los selectores son fundamentales en CSS, ya que **permiten dirigir los estilos a elementos específicos en una página web**, lo que brinda un mayor control sobre el diseño y la apariencia de los elementos.

Los selectores funcionan mediante la **especificación de un patrón** que coincide con uno o varios elementos en el documento HTML. Una vez que se seleccionan los elementos, se pueden aplicar reglas de estilo para cambiar la apariencia de esos elementos, como el color, el tamaño de fuente, el margen, el relleno, etc.

El selector por etiqueta: El selector de etiqueta en CSS es uno de los selectores más simples y básicos. Permite seleccionar todos los elementos de un tipo específico dentro de un documento HTML y aplicar estilos a esos elementos.

El selector de etiqueta utiliza el nombre de la etiqueta HTML como selector. Por ejemplo, para seleccionar todos los elementos de párrafo <p> en un documento HTML, simplemente se utiliza el selector "p".

```
p {  
  color: blue;  
  font-size: 16px;  
}
```

En este caso, se seleccionarán **todos los elementos de párrafo <p>** en el

documento y se aplicarán los estilos definidos. En este ejemplo, los párrafos se mostrarán en color azul y con un tamaño de fuente de 16 píxeles.

El selector de etiqueta es muy útil cuando se desea aplicar estilos a todos los elementos de un tipo específico de manera uniforme. Por ejemplo, si deseas aplicar estilos a todos los encabezados `<h1>` en tu página, simplemente puedes utilizar el selector "h1".

Si necesitamos ser un poco más específicos, podemos preguntar por elementos dentro de otros elementos. Esto se hace dejando un espacio entre los nombres de ambos. Eso significa que el de la derecha está dentro de la izquierda. No hay límite en este encapsulamiento. Si quisiéramos seleccionar todos los párrafos, pero sólo del elemento "div", podríamos hacer lo siguiente:

```
div p {  
  color: blue;  
  font-size: 16px;  
}
```

Las Clases y los ID

Una **clase** es un atributo de un elemento html y su valor será el nombre que nosotros definamos para ella. Es importante recordar que es conveniente elegir un nombre que nos oriente acerca de lo que hace dicha clase. Por ejemplo, si quisiéramos definir un estilo para centrar diferentes elementos, podríamos llamar a esa clase "center".

Estos atributos son acumulables, por lo que puedo ponerle todas las clases que quiera a un mismo elemento. Además, **son reutilizables**, ya que puedo usarlas en distintos elementos. Las clases se aplican a cualquier elemento HTML, mediante **el atributo class**. Debemos seguir ciertas normas al elegir el nombre: que no contengan espacios y que el primer carácter no sea un número o un carácter especial.

```
<p class="ejemplo">Lorem ipsum dolor sit amet.</p>
```

Selector por clase: Para seleccionar elementos por clase, se debe utilizar el símbolo de punto (.) seguido del nombre de la clase. Por ejemplo, si tienes elementos con la clase "ejemplo", el selector sería **".ejemplo"**. Asegúrate de incluir el punto antes del nombre de la clase.

Para aplicar estilos a los elementos seleccionados por clase, simplemente define las reglas CSS correspondientes para el selector. Por ejemplo:

```
.ejemplo {  
  color: blue;  
  font-size: 16px;  
}
```

En este caso, todos los elementos con la clase "ejemplo" tendrán el color de texto azul y un tamaño de fuente de 16 píxeles.

Por otro lado, además de las clases, tenemos otro atributo en HTML que nos permite identificar elementos: el **ID**. Mientras que el atributo class nos permite identificar grupos, **el ID nos permite nombrar objetos únicos. Ningún ID se puede repetir en un mismo sitio** y, en general, reservamos su uso para algún ítem muy importante dentro de la página web o para elementos a los que queramos poder acceder con un vínculo, ya que el ID nos permite vincular a éstos.

Para agregar un ID, seguimos las mismas reglas que para la clase:

```
<header id="miElemento">  
  <h1>Bienvenidos</h1>  
</header>
```

Selector por ID: para seleccionar un elemento específico por su ID, se debe utilizar el símbolo de almohadilla (#) seguido del nombre del ID. Por ejemplo, si tienes un elemento con el ID "miElemento", el selector sería "#miElemento". Asegúrate de incluir el almohadilla antes del nombre del ID.

Al igual que con los selectores por clase, puedes aplicar estilos al elemento seleccionado por ID definiendo las reglas CSS correspondientes para el selector. Por ejemplo:

```
.miElemento {  
  color: blue;  
  font-size: 16px;  
}
```

Una aclaración importante

Si bien es posible, no es recomendable utilizar **la selección por ID** a menos que sea absolutamente necesario. **Los ID se deben utilizar para facilitar la navegación**, dentro de las etiquetas <a>. Si solamente necesitamos agregar un

estilo de CSS, en lugar de usar un ID se recomienda aplicar una clase única a ese elemento.

El modelo de Cajas

El **modelo de cajas** es uno de los conceptos fundamentales en CSS que describe cómo se representa y se estructura visualmente un elemento HTML en una página web. **Cada elemento HTML se considera una "caja" que tiene varias propiedades, como el contenido, el relleno, el borde y el margen.**

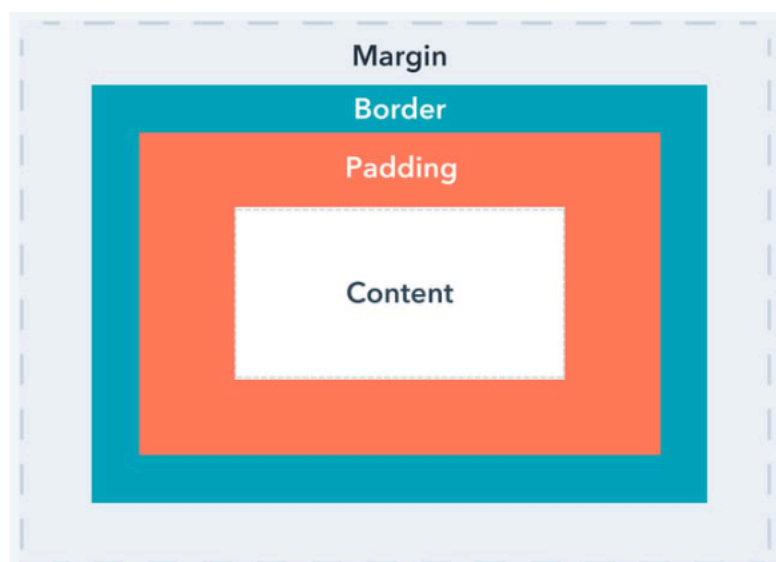
Las diferentes partes del modelo de cajas son:

Contenido (Content): Es el área que contiene el contenido real del elemento, como texto, imágenes, videos u otros elementos HTML.

Relleno (Padding): Es el espacio entre el contenido y el borde del elemento. El relleno proporciona un espacio adicional dentro del elemento, lo que afecta la distancia entre el contenido y el borde.

Borde (Border): Es una línea que rodea el contenido y el relleno del elemento. El borde puede tener diferentes estilos, anchos y colores.

Margen (Margin): Es el espacio exterior al elemento, que separa al elemento de otros elementos en la página. El margen crea un espacio vacío alrededor del elemento.



El modelo de cajas en CSS permite aplicar estilos y controlar el tamaño, la posición y el espaciado de los elementos en una página web. Al utilizar

propiedades como el ancho, la altura, el relleno, el borde y el margen, puedes ajustar y personalizar la apariencia de los elementos.

Es importante tener en cuenta que el tamaño total de un elemento en la página web se calcula sumando el ancho del contenido, el relleno, el borde y el margen. Por lo tanto, al establecer estas propiedades, debes considerar el tamaño total que deseas que ocupe el elemento en el diseño de tu página.

El modelo de cajas es esencial para el diseño y la maquetación en CSS, ya que proporciona una forma de controlar y organizar el diseño de los elementos en una página web.

Márgenes (margin):

Los márgenes son una propiedad de CSS que sirven para separar los elementos HTML entre sí y del borde exterior del contenedor que los rodea. **Los márgenes proporcionan espacio adicional alrededor de un elemento** y se utilizan para crear espacios en blanco o separación visual entre los elementos.

Existen diferentes formas de especificar los márgenes en CSS:

Margen individual: Se puede especificar márgenes individualmente para cada lado del elemento utilizando las propiedades **margin-top**, **margin-right**, **margin-bottom** y **margin-left**. Por ejemplo:

```
.elemento {  
  margin-top: 10px;  
  margin-right: 20px;  
  margin-bottom: 10px;  
  margin-left: 20px;  
}
```

En este caso, se establecen márgenes de 10 píxeles en la parte superior e inferior, y márgenes de 20 píxeles en los lados derecho e izquierdo del elemento.

Margen abreviado: También se puede utilizar la propiedad **margin** para especificar los márgenes de manera abreviada. puedes proporcionar uno, dos, tres o cuatro valores separados por espacios. Los valores se aplicarán en el orden de arriba, derecha, abajo e izquierda. Por ejemplo:

```
.elemento {  
  margin: 10px 20px 10px 20px; //arriba, derecha, abajo e izquierda
```

```

}

.elemento {
  margin: 10px 20px; //arriba y abajo, derecha e izquierda
}

.elemento {
  margin: 10px; //en todos los puntos por igual
}

```

Margen automático: Utilizar `margin: auto` permite centrar horizontalmente un elemento dentro de su contenedor. Esto ajustará automáticamente los márgenes laterales para que el elemento esté centrado.

Los márgenes también pueden tener valores negativos, lo que permite que los elementos se superpongan entre sí o se ajusten a un diseño específico. Es importante tener en cuenta que los márgenes se acumulan entre elementos adyacentes. Esto significa que si hay varios elementos uno al lado del otro, los márgenes se sumarán, lo que puede afectar el espacio total entre ellos.

Los márgenes son una herramienta útil en CSS para controlar la separación entre elementos y lograr el diseño deseado en una página web. Al utilizar los márgenes de manera efectiva, puedes crear espacios claros y organizados en tu diseño.

Relleno (Padding):

El padding es otra propiedad de CSS que se utiliza para agregar espacio adicional dentro de los límites de un elemento HTML. A diferencia de los márgenes que proporcionan espacio externo, el padding crea espacio interno alrededor del contenido de un elemento.

El padding se utiliza para separar el contenido del borde del elemento y se aplica a los cuatro lados: superior, derecho, inferior e izquierdo. Al igual que con los márgenes, hay diferentes formas de especificar el padding en CSS:

Padding individual: Se puede especificar el padding individualmente para cada lado del elemento utilizando las propiedades `padding-top`, `padding-right`, `padding-bottom` y `padding-left`. Por ejemplo:

```

.elemento {
  padding-top: 10px;
  padding-right: 20px;
}

```

```
padding-bottom: 10px;
padding-left: 20px;
}
```

En este caso, se establece un padding de 10 píxeles en la parte superior e inferior, y un padding de 20 píxeles en los lados derecho e izquierdo del elemento.

Padding abreviado: También se puede utilizar la propiedad padding para especificar el padding de manera abreviada. Al igual que con los márgenes abreviados, puedes proporcionar uno, dos, tres o cuatro valores separados por espacios, que se aplicarán en el orden de arriba, derecha, abajo e izquierda. Por ejemplo:

```
.elemento {
padding: 10px 20px 10px 20px; //arriba, derecha, abajo e izquierda
}

.elemento {
padding: 10px 20px; //arriba y abajo, derecha e izquierda
}

.elemento {
padding: 10px; //en todos los puntos por igual
}
```

El padding es útil para crear espacios internos alrededor del contenido de un elemento y puede ayudar a mejorar la legibilidad y el diseño de una página web. Al utilizar el padding de manera efectiva, se puede ajustar el espaciado y la separación dentro de los elementos para lograr el aspecto deseado en tu diseño.

Bordes (Border)

La propiedad border en CSS se utiliza para establecer el borde alrededor de un elemento HTML. **El borde puede ser una línea sólida, punteada, en relieve, con bordes redondeados u otras variaciones dependiendo de las propiedades que se le apliquen.**

Existen varias propiedades relacionadas con el borde que se pueden utilizar:

- **border-width:** Establece el ancho del borde. Se puede especificar un valor en píxeles, puntos, porcentaje o utilizando palabras clave como "thin", "medium" o "thick".

- **border-style**: Define el estilo del borde, como **"solid"** (sólido), **"dotted"** (punteado), **"dashed"** (guiones), **"double"** (doble línea), **"groove"** (grabado), **"ridge"** (relieve) y otros.
- **border-color**: Establece el color del borde. puedes utilizar un valor de color en formato hexadecimal, de palabras clave o de otros formatos.

Estas tres propiedades se pueden combinar en la propiedad abreviada **border**, donde se especifica el ancho, el estilo y el color del borde en ese orden. Por ejemplo:

```
.elemento {  
  border: 10px solid red; }
```

Este código establecería un borde de 10 píxeles de ancho, sólido y de color rojo alrededor del elemento con la clase **.elemento**.

Además de estas propiedades básicas, también puedes controlar los bordes individuales utilizando propiedades más específicas:

border-top, **border-right**, **border-bottom**, **border-left**: Permiten establecer el estilo, ancho y color de cada borde de forma individual.

border-radius: Se utiliza para especificar el radio de los bordes redondeados. puedes establecer un valor en píxeles o porcentaje para definir la curvatura de los bordes.

```
.elemento {  
  border: 10px solid red;  
  border-radius: 10px;  
}
```

El uso de la propiedad **border** y sus propiedades relacionadas te permite personalizar y dar estilo a los bordes de los elementos HTML en tu página web. Esto puede ayudar a resaltar y dar estructura visual a los elementos en tu diseño.

```
.elemento {  
  border: 10px solid red;  
  border-bottom-right-radius: 20px;  
  border-radius: 10px;
```

```
}
```

En este caso, se establece un borde sólido de 2 píxeles de ancho con color negro al elemento .elemento. El radio de borde de 10 píxeles se aplica a la esquina superior izquierda, mientras que el radio de borde de 20 píxeles se aplica a la esquina inferior derecha del elemento.

Estilos más utilizados (fuentes, líneas, cajas, etc...)

La tipografía es un elemento importante en el diseño de una página web, ya que puede influir en la legibilidad, el estilo y la apariencia general del contenido textual. En CSS, hay varias propiedades que se utilizan para aplicar estilos a la tipografía. Aquí tienes algunas propiedades comunes relacionadas con la tipografía:

1. **font-family:** Esta propiedad se utiliza para especificar el tipo de fuente que se utilizará para el texto. Se puede utilizar nombres de fuentes específicos o una lista de fuentes genéricas, como "Arial", "Helvetica", "Times New Roman", etc. También es posible definir múltiples fuentes en caso de que una no esté disponible en el dispositivo del usuario.
2. **font-size:** Permite establecer el tamaño de la fuente. Se puede utilizar unidades de píxeles, puntos, porcentaje, em o rem para especificar el tamaño. Al elegir un tamaño de fuente, es importante considerar la legibilidad en diferentes dispositivos y tamaños de pantalla.
3. **font-weight:** Esta propiedad controla el grosor o la negrita del texto. Se puede utilizar valores como "normal", "bold", "bolder", etc. para establecer diferentes niveles de peso en la fuente.
4. **line-height:** Se utiliza para establecer la altura de línea, es decir, el espacio vertical entre líneas de texto. Un valor adecuado para line-height puede mejorar la legibilidad y el flujo visual del texto.
5. **text-align:** Esta propiedad se utiliza para alinear el texto dentro de su contenedor. Se puede alinear el texto al centro, a la izquierda, a la derecha o justificarlo.

Además de estas propiedades, existen otras que pueden afectar la tipografía, como `text-decoration` para agregar decoraciones al texto (subrayado, tachado, etc.), `text-transform` para cambiar el caso del texto (mayúsculas, minúsculas, etc.), y `letter-spacing` para ajustar el espaciado entre caracteres.

Es importante elegir una combinación de fuentes, tamaños y estilos de tipografía que se adapte al estilo y propósito de tu sitio web, y que proporcione una experiencia de lectura cómoda y agradable para los usuarios. También se puede utilizar fuentes personalizadas mediante la integración de fuentes web, como Google Fonts, para añadir un toque único a tu diseño tipográfico.

Agregar tipografías de Google fonts

Todos los navegadores cuentan con fuentes por defecto, pero seguramente vamos querer personalizar nuestra página eligiendo las fuentes que más nos gusten. Además, debemos asegurarnos de que, en todos los navegadores, nuestra página se vea con esa misma fuente que seleccionamos. Una solución para esto es utilizar Google Fonts, que es una herramienta gratuita. Vamos a ver cómo añadirla a nuestra página.

Si buscás una fuente específica y tenés una idea de su nombre, puedes usar la barra de búsqueda (Search 🔍) de la parte superior izquierda y escribir allí el nombre, y una vez que la encontraste, puedes seleccionarla haciendo clic.

Se abrirá una nueva ventana con toda la familia de esa fuente, donde vas a poder elegir qué tipografía se ajusta mejor a tu diseño. Cuando ya te decidiste, puedes hacer clic en el signo "+" de la derecha, donde dice "Select this style".

Se va a abrir un pop-up con la información de la fuente. Para embeber la fuente tenés que:

- Copiar el link de la fuente en el "head" de tu html.
- Copiar la regla CSS y pegarla dentro de tu archivo CSS.

Box-shadow

La propiedad **box-shadow** en CSS permite agregar sombras a los elementos de tu página web. Puedes utilizar esta propiedad para crear efectos de profundidad y resaltar visualmente los elementos en tu diseño. La sintaxis de box-shadow es la siguiente:

```
box-shadow: [desplazamiento horizontal] [desplazamiento vertical]
```

```
[desenfoque] [extensión] [color];
```

```
//ejemplo:
```

```
.elemento {  
  box-shadow: 2px 2px 4px 2px rgba(0, 0, 0, 0.3);  
}
```

- ⇒ **Desplazamiento horizontal:** Especifica la distancia horizontal desde el elemento donde se mostrará la sombra. Un valor positivo desplaza la sombra hacia la derecha, mientras que un valor negativo la desplaza hacia la izquierda.
- ⇒ **Desplazamiento vertical:** Indica la distancia vertical desde el elemento donde se mostrará la sombra. Un valor positivo desplaza la sombra hacia abajo, mientras que un valor negativo la desplaza hacia arriba.
- ⇒ **Desenfoque (opcional):** Controla el nivel de borrosidad de la sombra. Un valor mayor crea una sombra más difuminada, mientras que un valor de 0px produce una sombra nítida y definida.
- ⇒ **Extensión (opcional):** Permite ampliar o reducir el tamaño de la sombra. Un valor mayor aumenta el tamaño de la sombra, mientras que un valor menor la reduce.
- ⇒ **Color (opcional):** Especifica el color de la sombra. puedes utilizar valores de color en formato hexadecimal, RGB o nombres de colores predefinidos.

Flexbox

Flexbox es un módulo de diseño en CSS que proporciona una forma flexible y eficiente de organizar y distribuir elementos en un contenedor. Con Flexbox, puedes crear diseños de cajas flexibles y receptivos que se adaptan automáticamente a diferentes tamaños de pantalla y orientaciones.

El modelo de caja flexible se basa en dos conceptos clave: el contenedor flex (flex container) y los elementos flexibles (flex items).

Para utilizar Flexbox, debes seguir estos pasos:

Establecer el contenedor flex:

- **Aplica la propiedad display:** flex; al contenedor que deseas convertir en un contenedor flex. Esto define un nuevo contexto de formato flexible para sus elementos secundarios.

Configurar la dirección y el flujo:

- Utiliza la propiedad **flex-direction** para establecer la dirección de los elementos secundarios dentro del contenedor flex. Puede ser row (horizontal), column (vertical), row-reverse o column-reverse.
- Utiliza la propiedad **flex-wrap** para permitir que los elementos secundarios se envuelvan en múltiples líneas si no caben en una sola línea.

Controlar la distribución y el espacio:

- Utiliza la propiedad **justify-content** para alinear los elementos secundarios a lo largo del eje principal del contenedor flex. Puede ser flex-start, flex-end, center, space-between, space-around o space-evenly.
- Utiliza la propiedad **align-items** para alinear los elementos secundarios a lo largo del eje transversal del contenedor flex. Puede ser flex-start, flex-end, center, stretch o baseline.
- Utiliza la propiedad **align-content** para controlar el espacio entre las líneas de elementos secundarios si hay varias líneas dentro del contenedor flex.

Estilizar los elementos secundarios:

- Aplica propiedades específicas a los elementos secundarios, como **flex-grow**, **flex-shrink** y **flex-basis**, para controlar cómo se distribuye el espacio disponible entre ellos.
- Utiliza la propiedad **order** para cambiar el orden de los elementos secundarios dentro del contenedor flex.

Flexbox ofrece una forma poderosa y eficiente de crear diseños flexibles y receptivos sin depender tanto de floats, posicionamiento absoluto o hacks de diseño. Permite una mayor facilidad en el diseño de interfaces, alineación de elementos y control del espacio dentro de un contenedor.

Buenas prácticas al construir una hoja de estilos 🙌

Al construir una hoja de estilos (CSS), es importante seguir algunas buenas

prácticas para mantener el código organizado, legible y fácil de mantener. Aquí tienes algunas recomendaciones:

```
/* Comentarios para documentación */
/* Estilos globales */

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
}

/* Nomenclatura consistente y descriptiva */

.header {
  background-color: #333;
  color: #fff; padding: 10px;
}

/* Evitar selectores demasiado específicos */

.container {
  width: 960px;
  margin: 0 auto;
}

/* Mantener la especificidad bajo control */

.section-title {
  font-size: 24px;
}

/* Reutilización de estilos */

.button {
  display: inline-block;
  padding: 8px 12px;
  background-color: #007bff;
  color: #fff;
  text-decoration: none;
}

/* Agrupar estilos relacionados */
```

```
.nav {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}  
  
.nav li {  
  display: inline-block;  
  margin-right: 10px;  
}  
  
/* Mantener el archivo CSS ordenado */  
/* Estilos paraespecíficos una página o componente */
```

1. **Utilizar comentarios:** Agrega comentarios en tu código CSS para explicar su propósito y proporcionar documentación. Esto ayuda a otros desarrolladores (y a ti mismo en el futuro) a comprender rápidamente el propósito de ciertas reglas y facilita la tarea de hacer cambios o modificaciones.
2. **Nomenclatura consistente:** Utiliza una nomenclatura coherente y descriptiva para nombrar tus clases y selectores. Esto ayuda a comprender el propósito y el contexto de cada estilo. puedes utilizar convenciones como BEM (Block Element Modifier) o SMACSS (Scalable and Modular Architecture for CSS) para estructurar y nombrar tus estilos de manera consistente.
3. **Evitar selectores demasiado específicos:** Evita el uso de selectores demasiado específicos y anidados excesivamente. Esto puede complicar el mantenimiento y la comprensión del código. Opta por selectores más generales y utiliza la cascada de CSS para aplicar estilos específicos según sea necesario.
4. **Mantener la especificidad bajo control:** Controla la especificidad de tus estilos para evitar conflictos y dificultades en la aplicación de estilos. Evita el uso excesivo de selectores de ID y trata de utilizar clases y selectores de tipo en su lugar.
5. **Reutilizar estilos:** Busca oportunidades para reutilizar estilos en lugar de duplicar código. Utiliza clases y selectores comunes para aplicar estilos similares a varios elementos en lugar de escribir reglas separadas para cada uno.

6. **Agrupar estilos relacionados:** Agrupa estilos relacionados o similares para una mayor legibilidad y organización. Esto facilita la identificación de estilos relacionados y su mantenimiento posterior.
7. **Mantener el archivo CSS ordenado:** Organiza tu archivo CSS de manera lógica y estructurada. puedes dividirlo en secciones basadas en componentes, características o páginas específicas. Además, mantén una estructura de carpeta ordenada para tus archivos CSS si tienes un proyecto más grande.
8. **Utilizar herramientas de construcción y preprocesadores:** Considera el uso de herramientas de construcción, como Gulp o Grunt, y preprocesadores de CSS, como Sass o Less, para optimizar y organizar tu flujo de trabajo de desarrollo CSS. Estas herramientas te permiten automatizar tareas repetitivas y facilitan la escritura de código más eficiente y modular.

Al seguir estas buenas prácticas, podrás crear hojas de estilos CSS más mantenibles, escalables y fáciles de trabajar, lo que facilitará el desarrollo y el mantenimiento de tu proyecto a largo plazo.

Manejo de assets e imágenes. Conociendo rutas absolutas y relativas.

Cuando trabajas con **assets** e **imágenes** en una página web, es importante comprender el manejo de rutas absolutas y relativas para acceder y mostrar correctamente los archivos. Aquí tienes una explicación sobre estos conceptos:

Rutas absolutas

Las rutas absolutas **especifican la ubicación completa** del archivo desde la raíz del sistema de archivos o el dominio. Por ejemplo, una ruta absoluta para una imagen en un sitio web podría ser: `/img/logo.png`. Esta ruta comienza desde la raíz del dominio o la estructura de archivos y se utiliza para acceder al archivo desde cualquier ubicación en el sitio web.

Las rutas absolutas son útiles cuando necesitas hacer referencia a recursos que se encuentran en ubicaciones fijas y permanentes en el servidor, como imágenes, archivos CSS o scripts.

Si tu sitio web está en el dominio **https://www.example.com**, puedes hacer referencia a las imágenes utilizando rutas absolutas desde la raíz del dominio

- ``

Estas rutas funcionarán correctamente desde cualquier ubicación en tu sitio web, ya que comienzan desde la raíz del dominio.

Rutas relativas

Las rutas relativas se definen en relación con la ubicación del archivo HTML actual. En lugar de especificar la ruta completa desde la raíz, **las rutas relativas utilizan referencias relativas a la ubicación actual del archivo HTML.**

Hay dos tipos comunes de rutas relativas:

Rutas relativas a la misma carpeta: Por ejemplo, si tienes una imagen llamada `imagen.jpg` en la misma carpeta que tu archivo HTML, puedes hacer referencia a ella utilizando `imagen.jpg` o `./imagen.jpg`.

Rutas relativas a una carpeta padre: Si tienes una estructura de carpetas y necesitas hacer referencia a un archivo que se encuentra en una carpeta padre, puedes utilizar `../` para navegar hacia arriba en la estructura de carpetas. Por ejemplo, si el archivo HTML está en una carpeta "sitio" y la imagen está en una carpeta "img" en el mismo nivel que "sitio", puedes hacer referencia a la imagen utilizando `../img/imagen.jpg`.

Supongamos que el archivo **index.html** está en la raíz del proyecto. puedes hacer referencia a las imágenes utilizando rutas relativas a la ubicación actual del archivo HTML:

- ``
- ``

Estas rutas relativas funcionarán correctamente porque se refieren a los archivos `logo.png` y `background.jpg` que **se encuentran en la carpeta img** en el mismo nivel que el archivo `index.html`

Es importante considerar el contexto y la ubicación del archivo HTML al especificar rutas relativas. Esto te permite acceder a los assets e imágenes de forma correcta independientemente de la ubicación del archivo HTML. Asegúrate de utilizar las rutas adecuadas en los atributos `src` o `href` de tus elementos `<img>`, `<link>`, `<script>`, etc.

Recuerda que las rutas absolutas y relativas son conceptos relativos al servidor o al sistema de archivos en el que se encuentra tu sitio web. Por lo tanto, es fundamental comprender la estructura de tu proyecto y utilizar las rutas adecuadas para acceder a los assets e imágenes de manera correcta y consistente en tu página web.

Orden jerárquico de aplicación de reglas CSS y el peso asociado a las reglas. 📌

El orden jerárquico de aplicación de las reglas CSS y el peso asociado a las reglas son conceptos importantes para comprender cómo se aplican y priorizan los estilos en una página web.

1. Orden jerárquico de aplicación de las reglas CSS:

- Las reglas CSS se aplican siguiendo un orden jerárquico basado en la especificidad de los selectores y la cascada de estilos.
- En general, las reglas más específicas tienen prioridad sobre las reglas menos específicas. Esto significa que una regla definida con un selector más específico tendrá más peso que una regla definida con un selector más general.
- El orden de aparición de las reglas en el archivo CSS también juega un papel importante. Si tienes reglas conflictivas con el mismo nivel de especificidad, la última regla definida en el archivo CSS prevalecerá y se aplicará.

2. Peso asociado a las reglas CSS:

- Cada regla CSS tiene un peso asociado que determina su prioridad y cómo se aplicará en caso de conflictos.

- Los diferentes tipos de selectores tienen diferentes niveles de peso. Aquí hay una lista general de los selectores ordenados de menor a mayor peso:
 - Selectores de tipo y pseudoelementos (p, div, ::before, ::after, etc.).
 - Selectores de clase, atributo y pseudoclasses (.clase, [atributo], :hover, etc.).
 - Selectores de ID (#id).
 - Selectores de estilo en línea (atributo style dentro del elemento HTML).
- En caso de que haya conflictos entre reglas con el mismo nivel de especificidad, la regla con un selector más pesado prevalecerá y se aplicará.

Asegúrate de utilizar selectores apropiados y específicos cuando sea necesario y evita la sobrespecificación excesiva. Además, organiza tu CSS de manera coherente y comprensible, colocando las reglas en un orden lógico y asegurándote de que las reglas más específicas aparezcan después de las reglas más generales.

El concepto de cascada, especificidad y herencia son elementos fundamentales en CSS que determinan cómo se aplican y propagan los estilos a los elementos HTML.

Cascada (Cascading):

La cascada se refiere al proceso de **aplicar y combinar los estilos CSS** a los elementos HTML. Cuando se carga una página web, el navegador lee las reglas CSS y las aplica en un orden específico siguiendo las reglas de cascada.

La cascada sigue una serie de reglas para determinar qué estilo prevalece en caso de conflictos. Estas reglas se basan en el orden de aparición de las reglas, la especificidad de los selectores y el peso asociado a las reglas.

La cascada permite que los estilos se combinen y se apliquen de manera controlada y predecible, permitiendo la separación de la estructura HTML y la presentación visual de una página web.

Especificidad (Specificity):

La especificidad es un concepto que determina qué estilo tiene prioridad cuando hay reglas conflictivas que se aplican a un mismo elemento.

Cada selector CSS tiene un nivel de especificidad que se calcula en función de los selectores utilizados en la regla.

En caso de conflictos, la regla con la especificidad más alta prevalecerá y se aplicará. **La especificidad se calcula asignando un valor numérico a cada tipo de selector (ID, clase, tipo) y sumando estos valores para obtener la especificidad total.**

Herencia (Inheritance):

La herencia es el mecanismo por el cual los estilos se propagan desde un elemento padre a sus elementos hijos.

Algunas propiedades CSS son heredadas por defecto, lo que significa que los estilos aplicados a un elemento se transmiten automáticamente a sus elementos hijos, a menos que se especifique lo contrario.

Esto permite establecer estilos generales en **elementos de nivel superior y afectar a todos sus descendientes sin tener que aplicar explícitamente los estilos en cada elemento hijo.**

El entendimiento de estos conceptos es esencial para escribir estilos CSS eficientes y comprensibles. La cascada, la especificidad y la herencia trabajan en conjunto para determinar cómo se aplican y propagan los estilos en una página web, permitiendo una gestión y control adecuados de la presentación visual.

Inspeccionando estilos con las herramientas para desarrolladores en el navegador

Las herramientas para desarrolladores en los navegadores web son una herramienta poderosa para inspeccionar y modificar los estilos de una página web en tiempo real. **Estas herramientas te permiten visualizar y analizar los estilos aplicados a los elementos, identificar conflictos, probar cambios y depurar problemas relacionados con el diseño y la apariencia.**

Para abrir las herramientas para desarrolladores en la mayoría de los navegadores, simplemente haz clic derecho en cualquier parte de la página y selecciona "Inspeccionar" o "Inspeccionar elemento". También puedes acceder a ellas a través de las opciones de desarrollador en el menú del navegador.

Una vez abiertas las herramientas para desarrolladores, encontrarás una interfaz que muestra el HTML de la página y los estilos aplicados a cada elemento. Puedes navegar por el árbol HTML y seleccionar elementos específicos para ver sus estilos en la pestaña "Styles" o "Estilos".

En la pestaña de estilos, puedes ver las reglas CSS aplicadas a un elemento, incluyendo los selectores, las propiedades y los valores. También puedes editar los estilos directamente en las herramientas para ver los cambios en tiempo real en la página.

Además de inspeccionar los estilos, las herramientas para desarrolladores ofrecen funcionalidades adicionales como la capacidad de desactivar estilos, modificar el contenido de la página, simular diferentes tamaños de pantalla y dispositivos, y mucho más. Estas características son útiles para probar y depurar la apariencia de una página web en diferentes escenarios.

Práctica: Inspección y modificación de estilos con DevTools

Objetivo: Familiarizarte con el uso de las DevTools para analizar y ajustar estilos CSS en tiempo real.

1. Abre en tu navegador el archivo `index.html` del ejercicio anterior.
2. Haz clic derecho sobre un título, párrafo o imagen y selecciona **"Inspeccionar"**.
3. Observa el panel lateral:
 - En la sección **"Elements"**, ubica el código HTML.
 - En la pestaña **"Styles"**, revisa las reglas CSS aplicadas.
4. Modifica una propiedad, por ejemplo:
 - Cambia el color: `color: red;`

- Cambia el tamaño de fuente: `font-size: 22px;`
Los cambios se aplicarán **de inmediato en la página**.

5. Agrega una nueva regla desde el panel, por ejemplo:

`background-color: lightblue;`

Observa cómo el cambio afecta al diseño sin necesidad de editar el archivo CSS.

6. Prueba el modo **Responsive Design** (`Ctrl + Shift + M` en Chrome) y ajusta el ancho de pantalla para comprobar cómo responde el diseño.
7. Finalmente, **restaura los valores originales** en tu código fuente.

Responsividad

La responsividad se refiere a la **capacidad de una página web para adaptarse y verse bien en diferentes dispositivos y tamaños de pantalla**. Con el aumento de la diversidad de dispositivos móviles y la proliferación de pantallas de diferentes resoluciones, es fundamental que los sitios web sean responsivos para brindar una experiencia de usuario óptima.

En CSS, puedes lograr la responsividad utilizando técnicas como media queries y unidades de medida relativas. Las media queries te permiten aplicar estilos específicos según las características del dispositivo o el tamaño de la pantalla. puedes definir reglas CSS que se activarán o desactivarán dependiendo de las condiciones establecidas en la media query.

MÓVILES VERTICALES	MÓVILES HORIZONTAL TABLET VERTICAL	TABLET HORIZONTAL ESCRITORIOS NORMALES	ESCRITORIOS MUY ANCHOS
			
Estilos normales	<code>@media (min-width: 768px) {}</code>	<code>@media (min-width: 1024px) {}</code>	<code>@media (min-width: 1200px) {}</code>

El concepto de responsividad

La responsividad, también conocida como diseño web adaptable o responsive design en inglés, es el enfoque de diseño y desarrollo web que busca crear sitios web que se adapten y respondan a diferentes dispositivos y tamaños de pantalla. El objetivo de la responsividad es brindar una experiencia de usuario óptima, independientemente del dispositivo que se esté utilizando para acceder al sitio web.

En un sitio web responsivo, **los elementos y el diseño se ajustan automáticamente según el tamaño y las características del dispositivo**. Esto incluye factores como el ancho de la pantalla, la resolución, la orientación (vertical u horizontal) y la capacidad táctil del dispositivo.

La responsividad se logra mediante el uso de técnicas como media queries, unidades de medida relativas (porcentajes, unidades de vista) y diseño fluido. Estas técnicas permiten que los elementos del sitio web se redimensionen y reorganicen de manera inteligente para adaptarse al espacio disponible en la pantalla del dispositivo.

Algunos aspectos clave de la responsividad incluyen:

- 1. Diseño adaptable:** Los diseños responsivos se ajustan a diferentes tamaños y resoluciones de pantalla. Esto implica reorganizar los elementos, ajustar el tamaño de fuente y las imágenes, y ocultar o mostrar contenido según sea necesario.
- 2. Navegación intuitiva:** Los sitios web responsivos suelen tener una navegación simplificada y fácil de usar en dispositivos móviles, como menús desplegados o hamburguesas. Esto permite a los usuarios acceder fácilmente al contenido sin tener que hacer zoom o desplazarse demasiado.
- 3. Optimización del rendimiento:** Los sitios web responsivos también tienen en cuenta el rendimiento, optimizando el tamaño y la carga de los recursos para adaptarse a diferentes conexiones y velocidades de Internet. Esto ayuda a mejorar la velocidad de carga y la experiencia del usuario.
- 4. Pruebas multiplataforma:** Es fundamental realizar pruebas exhaustivas en diferentes dispositivos y tamaños de pantalla para asegurarse de que el sitio web se vea y funcione correctamente en todas las plataformas. Esto incluye dispositivos móviles, tablets y diferentes navegadores web.

La responsividad es esencial en el mundo actual, donde el acceso a Internet se realiza a través de una amplia variedad de dispositivos con diferentes tamaños y capacidades. Al crear sitios web responsivos, se proporciona una experiencia de usuario consistente y agradable, independientemente del dispositivo utilizado, lo que mejora la usabilidad y la satisfacción del usuario.

Tipos de dispositivos y orientaciones

En el contexto de la responsividad en diseño web, existen diversos tipos de dispositivos y orientaciones a tener en cuenta. Aquí hay una descripción de algunos de ellos:

Dispositivos de escritorio: Son computadoras de escritorio o portátiles que generalmente tienen pantallas más grandes y orientación horizontal. Estos dispositivos suelen ser utilizados con teclado y mouse.

Dispositivos móviles: Incluyen teléfonos inteligentes y tabletas. Tienen pantallas más pequeñas y se utilizan principalmente en orientación vertical, aunque algunos dispositivos permiten la rotación de la pantalla para adaptarse a la orientación horizontal.

Tablet: Es un dispositivo con pantalla táctil más grande que un teléfono inteligente pero más pequeño que una computadora portátil. Puede tener diferentes orientaciones, pero la más común es la vertical.

Teléfono inteligente: Es un dispositivo portátil con capacidades de comunicación, navegación por Internet y diversas aplicaciones. Tiene una pantalla más pequeña y se utiliza principalmente en orientación vertical.

Orientación vertical: Es cuando el dispositivo se encuentra en posición vertical, con la altura siendo mayor que el ancho. Es la orientación más común en dispositivos móviles y se utiliza principalmente para leer contenido largo.

Orientación horizontal: Es cuando el dispositivo se encuentra en posición horizontal, con el ancho siendo mayor que la altura. Esta orientación se utiliza a menudo para visualizar contenido amplio, como imágenes, videos y juegos.

Breakpoint

Es importante considerar estos diferentes tipos de dispositivos y orientaciones al diseñar y desarrollar un sitio web responsivo. El objetivo es garantizar que el contenido y el diseño se adapten y sean óptimos en cada situación, proporcionando una experiencia de usuario adecuada y atractiva en todos los dispositivos y orientaciones posibles.

Un breakpoint (punto de quiebre) **es un punto específico en el diseño responsivo de un sitio web en el que se aplica un conjunto de estilos diferentes según el tamaño de la pantalla del dispositivo**. Los breakpoints se definen mediante media queries y se utilizan para adaptar el diseño y el contenido a diferentes rangos de tamaño de pantalla.

Cuando se alcanza un breakpoint en la visualización de un sitio web, se activan los estilos específicos definidos en la media query correspondiente. Esto permite ajustar el diseño, el tamaño de fuente, la disposición de los elementos y otras propiedades para que se vean y funcionen de manera óptima en dispositivos de diferentes tamaños.

Breakpoints:

- **Para dispositivos móviles:** Se pueden establecer breakpoints para pantallas pequeñas, como teléfonos inteligentes, con una media query que se active para anchos de pantalla menores o iguales a 480 píxeles, por ejemplo. En este punto, se pueden aplicar estilos que optimicen la legibilidad y el espacio de los elementos.
- **Para tabletas:** Se pueden definir breakpoints para pantallas de tabletas con una media query que se active para anchos de pantalla entre 481 y 1024 píxeles. En este punto, se pueden ajustar los estilos para aprovechar el espacio adicional y mostrar más contenido en paralelo.
- **Para pantallas de escritorio:** Se pueden establecer breakpoints para pantallas de escritorio o portátiles con una media query que se active para anchos de pantalla mayores a 1024 píxeles. Aquí, se pueden aplicar estilos que aprovechen el espacio extra disponible y mejoren la disposición de los elementos en una pantalla más amplia.

El concepto Mobile First

El concepto de Mobile First (primero móvil) es una filosofía de diseño y desarrollo web que se centra en **priorizar y diseñar la experiencia del usuario para**

dispositivos móviles antes que para dispositivos de escritorio o pantallas más grandes.

En lugar de adaptar un diseño de escritorio para dispositivos móviles, el enfoque Mobile First busca crear una experiencia óptima para los usuarios que acceden al sitio web desde dispositivos móviles. Esto se debe a que el uso de dispositivos móviles para navegar por Internet ha aumentado significativamente en los últimos años.

El enfoque Mobile First implica las siguientes prácticas:

Diseño responsivo: El diseño se realiza de manera que el contenido y la estructura se adapten fluidamente a diferentes tamaños de pantalla, desde dispositivos móviles hasta pantallas de escritorio. Se utilizan media queries y otras técnicas para aplicar estilos específicos según las características del dispositivo.

Simplificación del diseño: Al comenzar con un enfoque móvil, se promueve un diseño minimalista y centrado en el contenido esencial. Se prioriza la legibilidad, la usabilidad y la navegación intuitiva en pantallas pequeñas.

Mejora progresiva: A medida que se amplía la resolución y el tamaño de la pantalla, se agregan gradualmente más elementos y funcionalidades al diseño. Esto asegura que los usuarios móviles tengan una experiencia fluida y eficiente, sin cargarles con elementos innecesarios.

Optimización de rendimiento: Se pone un fuerte énfasis en el rendimiento y la velocidad de carga en dispositivos móviles. Se utilizan técnicas de optimización, como la compresión de imágenes, la carga diferida de recursos y la minimización de solicitudes al servidor.

El enfoque Mobile First reconoce la importancia de ofrecer una experiencia de usuario fluida y de calidad en dispositivos móviles, teniendo en cuenta las limitaciones de pantalla más pequeñas, las conexiones de datos más lentas y los diferentes comportamientos de los usuarios en dispositivos móviles.

Al diseñar y desarrollar con el enfoque Mobile First, se crea un sitio web que se adapta a las necesidades y preferencias de los usuarios móviles, proporcionando una experiencia optimizada y satisfactoria. Luego, se puede ampliar y mejorar el diseño para dispositivos de mayor tamaño y funcionalidades adicionales.

Por ejemplo, puedes utilizar una media query para ajustar el tamaño de fuente en dispositivos móviles:


```
@media (max-width: 768px) {  
  body {  
    font-size: 14px;  
  }  
}
```

En este caso, la regla se aplicará solo si el ancho de la pantalla es igual o menor a 768 píxeles.

Además de las media queries, también puedes utilizar unidades de medida relativas como porcentajes y unidades de vista (vw, vh) para adaptar el tamaño y la posición de los elementos en función del tamaño de la pantalla.

Media Queries

Las media queries son una función de CSS que permite aplicar estilos específicos a una página web en función de las características del dispositivo o del entorno de visualización. Estas consultas de medios te permiten adaptar el diseño y los estilos de una página a diferentes tamaños de pantalla, resoluciones, orientaciones y capacidades del dispositivo.

La utilización de media queries es una técnica clave en el diseño responsivo para adaptar estilos y diseños según las características del dispositivo y el tamaño de la pantalla. A través de las media queries, puedes establecer reglas CSS específicas que se aplicarán sólo cuando se cumplan ciertas condiciones de los medios.

La sintaxis básica de una media query es la siguiente:

```
@media tipo-y-condición {  
  /* Reglas CSS para aplicar en la consulta de medios */  
}
```

- El "tipo" de la media query se refiere al medio o dispositivo en el que se aplicarán los estilos. Algunos ejemplos comunes son "screen" (pantallas), "print" (impresión) y "all" (todos los medios).
- La "condición" de la media query establece los criterios específicos para que se apliquen los estilos. Puedes utilizar diferentes características como el ancho de la pantalla (width), la orientación (orientation), la densidad de

píxeles (resolution) y muchas más.

puedes combinar múltiples condiciones en una media query utilizando operadores lógicos como and y or. Por ejemplo:

```
@media (min-width: 768px) and (max-width: 1024px){  
  /* Reglas CSS para aplicar en dispositivos de ancho entre 768px y 1024px  
  */  
}
```

En este caso, las reglas se aplicarán solo si el ancho de la pantalla está entre 768 y 1024 píxeles.

Las media queries te permiten crear diseños y estilos responsivos que se adapten a diferentes dispositivos y tamaños de pantalla. puedes utilizarlas para modificar el diseño, cambiar el tamaño de fuente, ocultar o mostrar elementos y realizar otras adaptaciones según las necesidades de visualización.

Para probar cómo se visualiza un sitio web en distintos dispositivos, existen varias opciones que puedes considerar:

Utiliza herramientas de desarrollo del navegador: Los navegadores modernos como Google Chrome, Mozilla Firefox y Safari ofrecen herramientas de desarrollo que permiten emular diferentes dispositivos y tamaños de pantalla. Estas herramientas te permiten ver cómo se verá y se comportará tu sitio web en diferentes entornos.

Prueba en dispositivos reales: Si tienes acceso a diferentes dispositivos como teléfonos inteligentes, tabletas y computadoras con pantallas de diferentes tamaños, puedes cargar tu sitio web en cada uno de ellos para ver cómo se adapta y se ve. Esto te dará una idea más precisa de la experiencia real del usuario en cada dispositivo.

Emuladores y simuladores de dispositivos: Existen herramientas de software y servicios en línea que ofrecen emuladores y simuladores de dispositivos. Estas herramientas te permiten probar tu sitio web en diferentes dispositivos virtuales, lo que puede ser útil si no tienes acceso a una amplia.

Es importante recordar que, además del tamaño de la pantalla, también debes considerar otros aspectos como la velocidad de conexión, las capacidades táctiles y las limitaciones de los navegadores en cada dispositivo. Realizar

pruebas en múltiples dispositivos y escenarios te ayudará a asegurar que tu sitio web se vea y funcione de manera óptima en todas las plataformas.

Cierre

En esta unidad, hemos explorado diversos aspectos del manejo de hojas de estilo (CSS) en el desarrollo web. Durante el recorrido, hemos aprendido los fundamentos de CSS, desde su utilidad hasta su integración con HTML. Hemos descubierto cómo aplicar estilos en línea, embebidos y mediante archivos externos, brindándonos flexibilidad y reutilización en la presentación de nuestros sitios web. Además, hemos explorado las diferentes formas de seleccionar elementos mediante referencias, clases e identificadores, permitiéndonos estilizar de manera precisa y eficiente.

Asimismo, hemos abordado el modelo de cajas en CSS, comprendiendo cómo funciona y cómo podemos manipular sus propiedades para lograr el diseño deseado. También hemos explorado estilos más utilizados, como fuentes, líneas y cajas, y hemos adquirido buenas prácticas para construir hojas de estilos organizadas y mantenibles.

Además, hemos adentrado en conceptos fundamentales como el orden jerárquico de aplicación de reglas CSS y el peso asociado a las mismas, lo cual nos brinda el control necesario para estilizar nuestros elementos correctamente. También hemos aprendido a inspeccionar estilos con las herramientas para desarrolladores en el navegador, facilitando la depuración y ajuste de nuestros estilos.

Finalmente, hemos explorado la responsividad en el diseño web, comprendiendo la importancia de adaptar nuestros sitios a diferentes dispositivos y orientaciones. Hemos conocido el concepto de Mobile First y la utilización de media queries para crear diseños responsivos. También hemos aprendido a probar nuestros sitios en distintos dispositivos, asegurando una experiencia de usuario óptima.

Referencias

MDN Web Docs. (s.f.). Diseño receptivo. Mozilla.

https://developer.mozilla.org/es/docs/Learn/CSS/CSS_layout/Responsive_Design

MDN Web Docs. (s.f.). Introducción al modelo de caja de CSS. Mozilla.

https://developer.mozilla.org/es/docs/Web/CSS/CSS_box_model/Introduction_to_the_CSS_box_model

MDN Web Docs. (s.f.). Conceptos básicos de Flexbox. Mozilla.

https://developer.mozilla.org/es/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox

¡Muchas gracias!

Nos vemos en la próxima lección

